

Offboarding Safely

The deletion guide · `cleanup_targets.py` and its six gates

`cleanup_targets.py` is the only script that deletes anything. It defaults to a dry run and will not remove a single item until you flip `DRY_RUN` and the item clears every gate below. This guide is how to run it without deleting the wrong thing.

The dependency gate is the one that matters most. It does *not* clear away whatever blocks a deletion — it stops, and sends the item to review. That's the behavior that keeps an active user's map from breaking when a departed user's layer disappears underneath it.

The flow

Dry run → a human reads the manifest → live run. On a live run, each item goes: **back up** → **dependency-check** → **delete** → **log**. Nothing skips the backup.

The six gates (and why each exists)

- **Public** — public items are never auto-deleted; they go to the review file. (Private is safe; public needs eyes.)
- **Faculty & keep-list** — faculty/staff (by email pattern) and anyone on `data/keep_list.csv` (co-ops, protected staff) are a hard keep, even if the spreadsheet's keep column missed them.
- **Dependencies** — if anything still points at an item, the delete halts and the item goes to review.
- **Backup** — every item is exported or downloaded to `data/outputs/backups/` first; if the backup fails, it is not deleted.
- **Notice + grace** — the owner must have been emailed and the 15-day window passed.
- **Confirmation** — a live run makes you type DELETE (turn off only for unattended Notebook runs).

The switches

```

DRY_RUN           = True      # look, don't touch. flip last.
PERMANENT_DELETE  = False     # leave False -> 14-day recycle bin
REQUIRE_CONFIRMATION = True   # False only for unattended runs
GRACE_DAYS        = 15
THROTTLE_SECONDS  = 0.5      # raise if you see 503s

```

The gate toggles (`EXCLUDE_PUBLIC`, `GUARD_FACULTY`, `HALT_ON_DEPENDENCIES`, `BACKUP_BEFORE_DELETE`, `REQUIRE_NOTICE_AND_GRACE`) all default on. Turning one off should be a deliberate, documented choice.

What it writes

- `data/outputs/cleanup_manifest.csv` — everything considered, with its disposition.
- `data/outputs/cleanup_results.csv` — the outcome of each item (deleted, skipped, backup_failed, error).
- `data/outputs/cleanup_review_needed.csv` — the public and dependency holds, for a human to handle.
- `data/outputs/cleanup_priority_updated.csv` — the input list with `deleted=TRUE` stamped on fully-cleaned owners.

Backups and restore

Hosted layers are exported to a file geodatabase; other items are downloaded as-is; the temporary export item is removed afterward so the backup doesn't eat org storage. To restore: pull from `data/outputs/backups/`, or from the recycle bin within 14 days for supported types.

Running it

```
# 1. dry run, then read the manifest + review file
python3 scripts/cleanup_targets.py

# 2. set DRY_RUN = False, then for real
python3 scripts/cleanup_targets.py
```

Before a live run — checklist

- ✓ Graduation status is on the list, back from IT
- ✓ Owners notified; `notice_date` filled; grace elapsed
- ✓ Dry-run manifest reviewed and signed off by a second person
- ✓ `PERMANENT_DELETE = False`
- ✓ You can reach `data/outputs/backups/` and the recycle bin if you need to undo